Mini-Cloud Billing Desk

Full-stack take-home: billing that stays correct when things go wrong

TypeScript · Node 24 · Fastify · PostgreSQL 17 · React 19 · Docker Compose

Every diagram is generated from `docs/diagrams/*.mmd` and drawn from the code as built.

What I built, and what I left out

Required

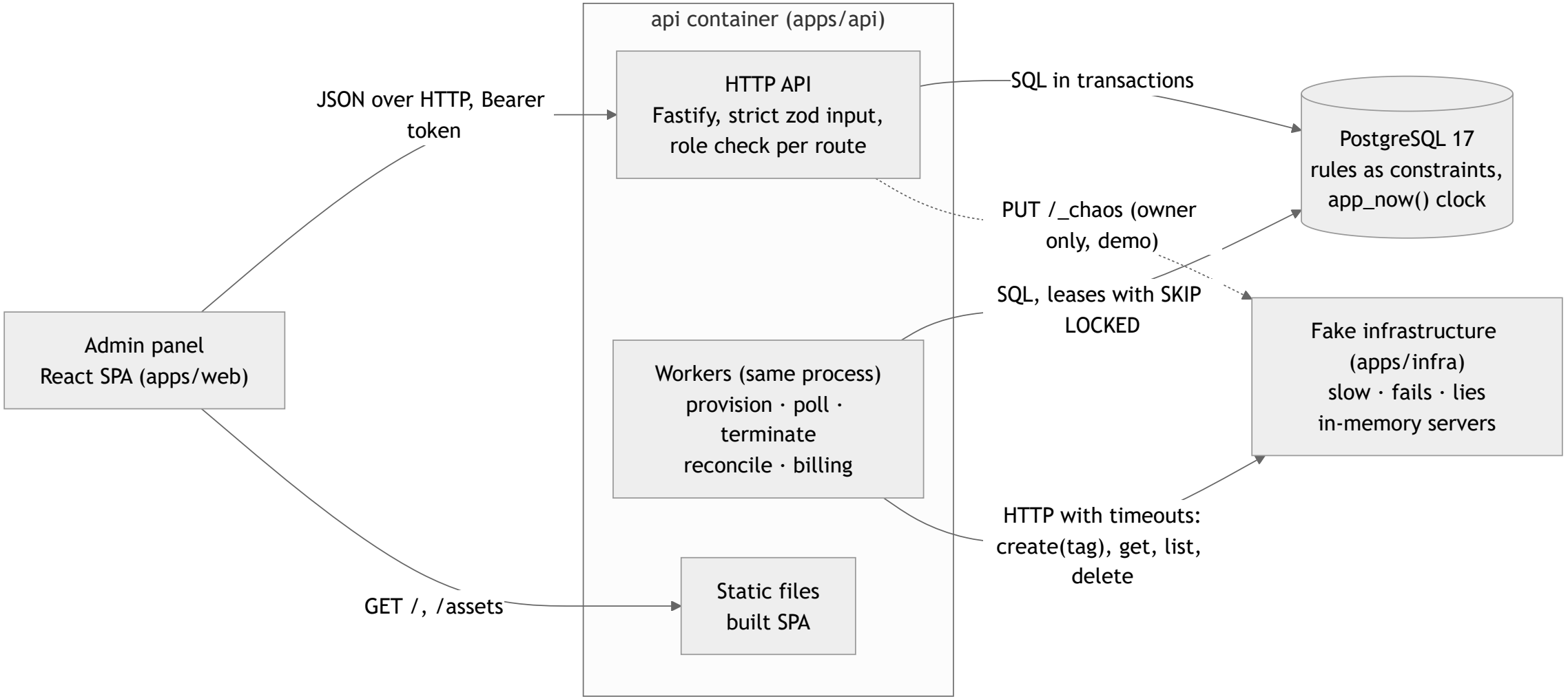
- Fake infrastructure: slow, unreliable, dishonest (configurable)
- Wallet as an append-only ledger; retry-safe top-ups
- Servers, IP pool, moves across customers, hourly billing
- Owner / support / viewer enforced by the server
- Strict input: unknown parameters are a 400
- Admin panel: URL state, stale-safe search, loading / empty / failed
- Testable clock that only moves forward
- Challenges 1-3, each with an automated test

Optional challenges

The infrastructure lies (drift repair)	built
Downtime: missed hours charged once	built
Restore within 24 h, old IP if free	built
One bad record does not stop billing	built
Price change from the next hour	built
Running out of money (notice, suspend, terminate)	not built

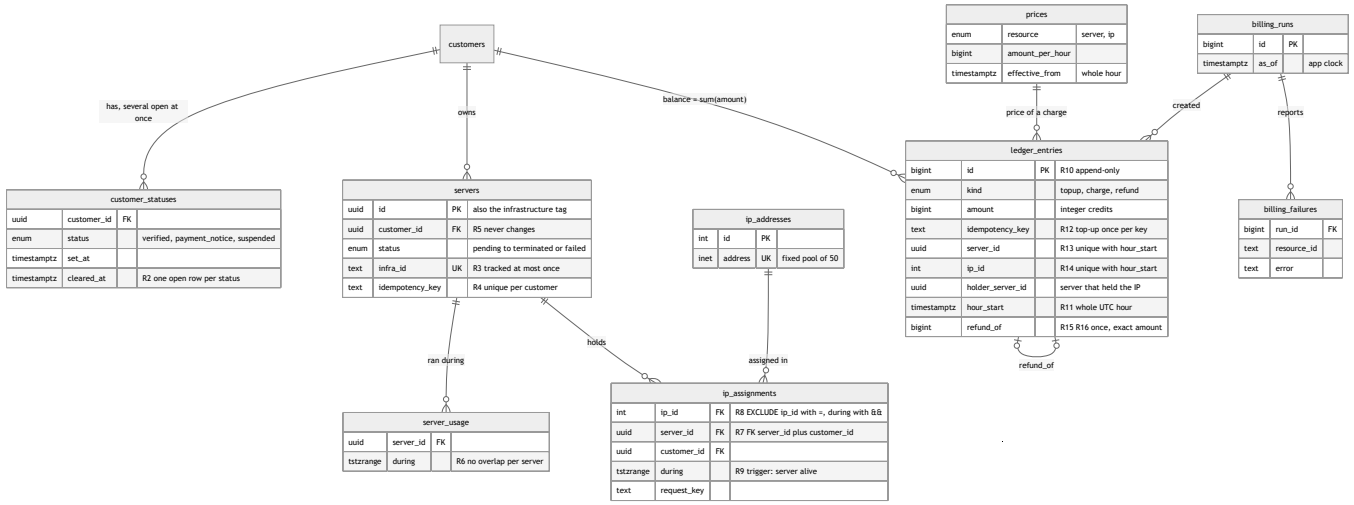
Statuses (verified, payment notice, suspended) exist and suspension blocks new servers, but no job moves a customer through them yet.

Parts of the system



One command starts db, infra and api; the api also serves the built panel. Workers run in the api process and use leases, so several api instances can run.

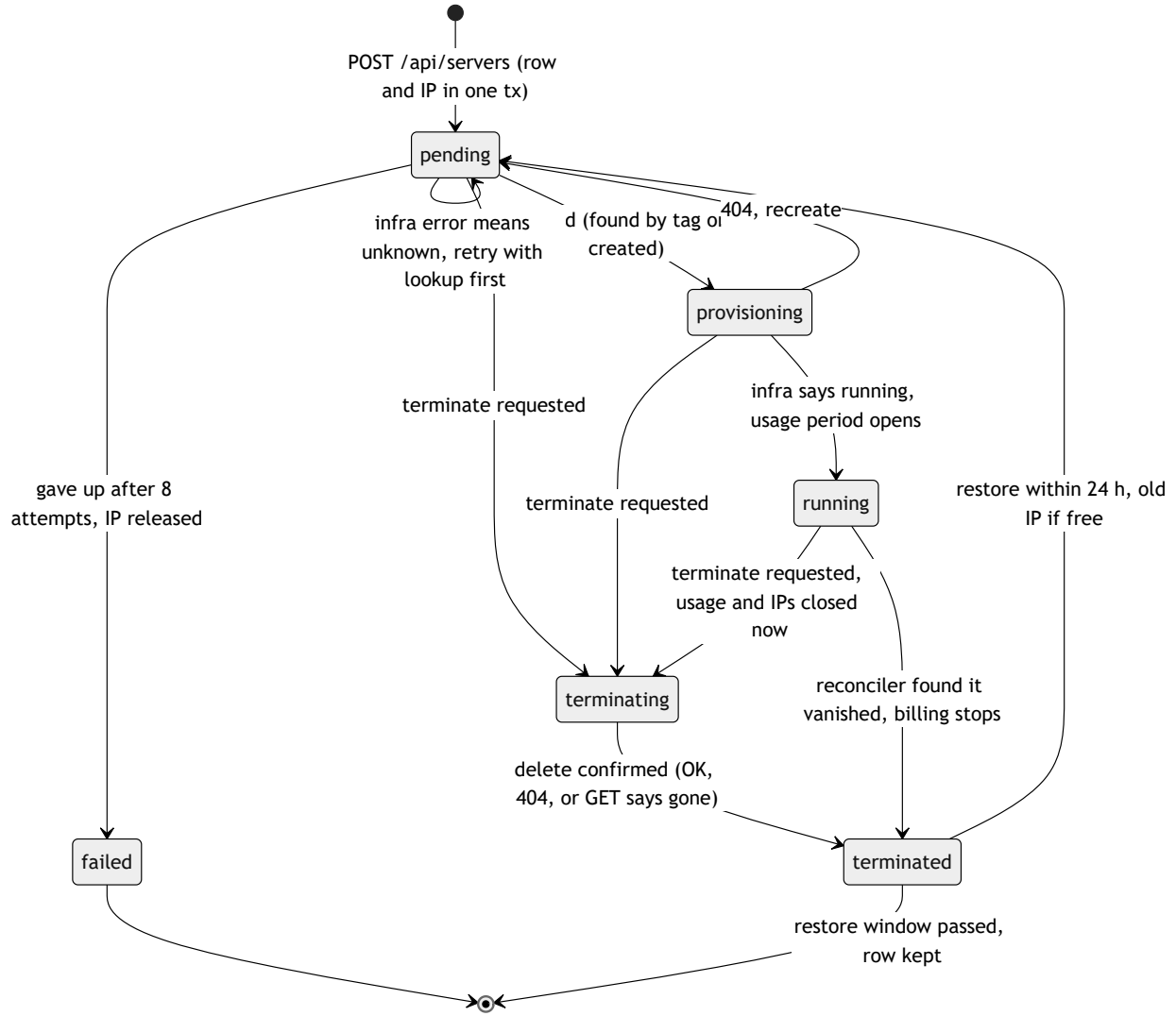
Data model: rules the database enforces



- **R13/R14** unique (server or IP, hour): a charge exists once
- **R8** `EXCLUDE (ip_id =, during &&)`: one holder per IP at any moment
- **R7** FK (server, customer): the payer of an IP is the owner of its server
- **R9** trigger: IPs only attach to alive servers (locks the server row)
- **R10** ledger is append-only (update / delete / truncate raise)
- **R11** a charge is one resource and one whole UTC hour
- **R12/R4** idempotency keys for top-ups and server creation
- **R15/R16** a charge is refunded once, by its exact amount
- **R3** an infrastructure server is tracked by at most one record
- **R1** the application clock only moves forward

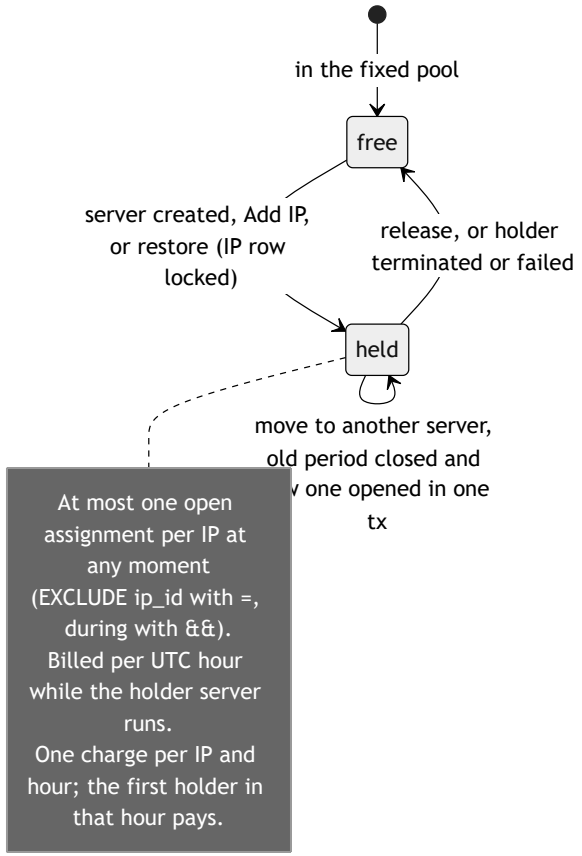
Money is `bigint` credits; all times are `timestamptz` in UTC.

Life cycle of a server



Billing follows `server_usage` periods, which open at "running" and close at the termination request, not at the infrastructure's confirmation.

Life cycle of an IP address



An IP is billed for the time it is attached to a server that runs: the intersection of the assignment period and the usage period.

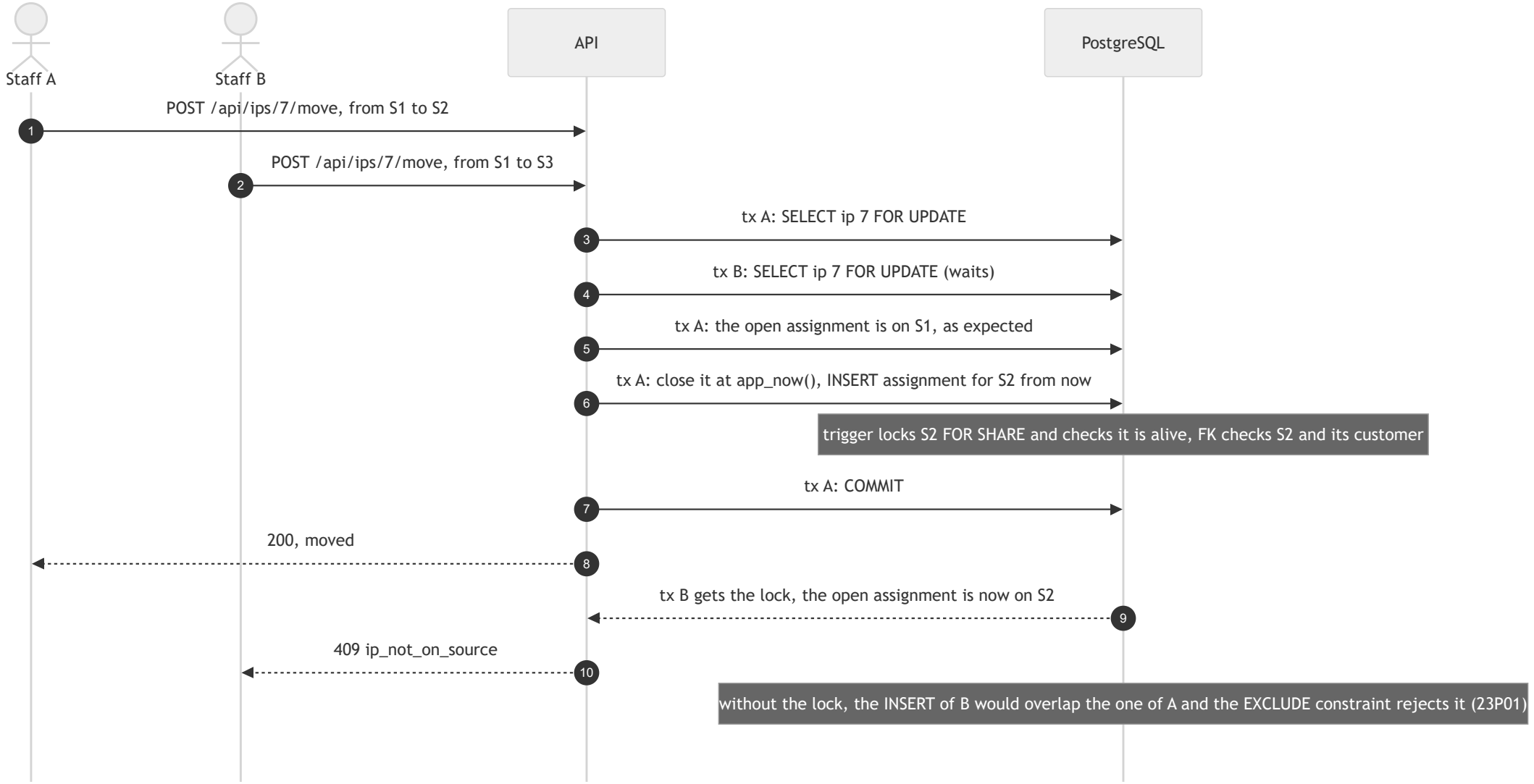
Creating a server when the infrastructure fails or lies



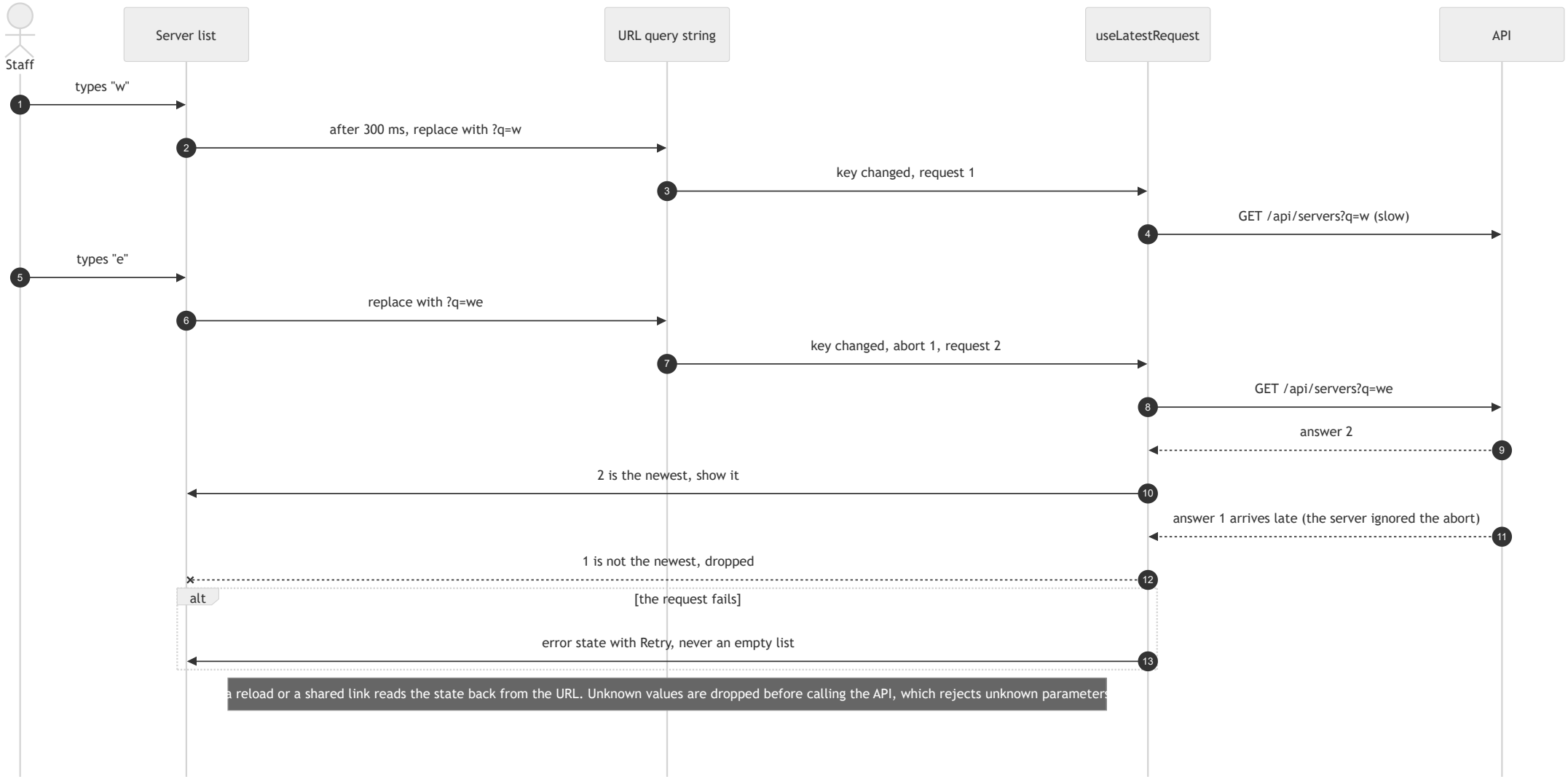
The billing run: concurrent, crashed, or late



Moving an IP, twice at the same moment



Searching in the UI



Main decisions and the alternatives I rejected

Decision	Why	Rejected
Catch-up billing: recompute due hours, insert missing ones	Retries, crashes, overlap and downtime all converge; no job state to lose	"Bill the last hour" cron job (gaps after downtime, duplicates on retry)
Unique index + `ON CONFLICT DO NOTHING`	The database decides exactly-once, under any interleaving	Check-then-insert, advisory lock as the only guard
Clock-aligned UTC hours, first holder pays a shared hour	One simple key per hour makes "one payer per IP-hour" a constraint	Hours relative to each start (two customers could pay overlapping hours)
IP history as ranges with an exclusion constraint	Overlap is impossible, history is kept for audits	A `server_id` column on the IP (no history, no DB guard)
Our id as the infrastructure tag; look up before create	An unclear create can always be resolved, never duplicated	Retrying create on error (double servers)
Raw SQL, no ORM; strict zod schemas	Constraints and locks stay visible and reviewable	ORM with implicit queries; JSON-schema with silent stripping
Sequence number + abort in one hook	Works even when the server ignores the abort	Abort only; a global data cache

Challenges: how, and which test proves it

Challenge	How	Test (all in the repo, run by `npm test`)
Exactly-once billing	Unique (resource, hour) + ON CONFLICT; one transaction per resource	`billing.test.ts`: two runs at once; crash halfway then rerun. *Seen failing* with check-then-insert: 280 charges instead of 150
One IP, one payer	Exclusion constraint on assignment ranges, IP row lock, expected source	`ip.test.ts`: 10 rounds of two simultaneous moves; DB refuses overlap; IP bounced between customers billed once per hour. Passes even with the app lock removed
Stale search	Sequence check + abort in `useLatestRequest`	`stale-search.test.tsx`: old answer resolved after the new one. *Seen failing* without the sequence check
Infrastructure lies	Tag lookup before create, confirmed deletes, reconciler	`workers.test.ts`: scripted lies, orphans, duplicates, 30%/30% random chaos
Downtime · bad record · price	Catch-up recompute · per-item transactions · price per hour row	`billing.test.ts`
Restore	Prefer the last IP if free, 24 h window	`ip.test.ts`
Roles, strict input, retries	Permission per route, strict schemas, idempotency keys	`http.test.ts`

54 automated tests (4 infrastructure, 44 API against a real PostgreSQL, 6 frontend). Regression notes with the observed failures: `docs/regression-evidence.md`.

What is not safe yet, and what I would do next

- **Billing cost grows with history.** Each run recomputes from all usage periods. Next: a per-resource "billed through" watermark and partitioned ledger.
- **Negative balances are allowed.** Charges are postpaid; nothing notifies or suspends yet. Next: the dunning flow with a recorded notice as a precondition (FK) for suspension.
- **A server lost by the infrastructure** is billed until the reconciler confirms it is gone (every 20 s in the Compose setup). Next: use the infrastructure's own stop time if it offers one.
- **Billing stops at the termination request**, even if the delete takes longer. A deliberate customer-friendly choice; we absorb the cost.
- **No monitoring.** Failed billing items and drift are stored and shown, but nothing alerts. Next: metrics and an alert on `billing_failures`.
- **Auth is minimal.** Bearer token in localStorage, no login rate limit, no password reset. Next: httpOnly cookie, rate limiting, SSO.
- **The fake infrastructure keeps state in memory.** Restarting it makes every server vanish; the reconciler then stops billing them (correct for the fake, noisy in a demo).
- **Search uses ILIKE.** Fine for this size; next: trigram indexes or keyset paging.
- **Docker Compose** was written but could not be run on my machine (Docker engine did not start); the same commands were run natively against PostgreSQL.

"You charged us for an IP we removed yesterday." Are they right?

1. Open **IP pool** → **the address** (`/ips/:id`). "Who held it" shows every assignment with its period; the removal is the `until` of their last row. `audit_log` (`ip.release`, `ip.move` or `server.terminate`) says who did it and when.
2. The same page lists every hourly charge of that IP with the payer and the server that held it.
3. Rule: the hour in which it was removed is charged (a started hour counts), later hours must belong to someone else or no one.
4. If a charge to them starts after that hour, they are right: refund it from the customer page (once, exact amount, enforced) and treat it as a bug.

```
-- charges to the customer after the hour they let go of the IP
select l.id, l.hour_start, l.amount
from ledger_entries l
where l.ip_id = $ip and l.customer_id = $customer
  and l.kind = 'charge'
  and l.hour_start >= date_trunc('hour',
      (select max(upper(during)) from ip_assignments
       where ip_id = $ip and customer_id = $customer), 'UTC')
      + interval '1 hour'
  and not exists (select 1 from ip_assignments a
       join server_usage u on u.server_id = a.server_id
       where a.ip_id = l.ip_id and a.customer_id = l.customer_id
         and a.during * u.during &&
             tstzrange(l.hour_start, l.hour_start + interval '1
hour'));
```

Empty result: the charges are correct, and the IP page shows them why.

Run it, test it, demo it

Start

`docker compose up --build` → `http://localhost:8080`

owner@ / support@ / viewer@example.com, passwords owner-pass / support-pass / viewer-pass

Test

`docker compose --profile test run --rm test`

or `npm test` with `TEST_DATABASE_URL` pointing at a PostgreSQL

Demo script

1. Servers: seeded servers go pending → running through the lying infra; Operations → drift lists any adoption or clean-up.
2. Search "acme" + status running; reload; the state stays.
3. Operations: +1 hour, run billing twice: the second run adds 0 charges.
4. Server page: move its IP to another customer; the IP page shows one payer per hour.
5. Sign in as support: no top-up button, and the API answers 403 anyway.
6. Set "calls that lie" to 50% and create servers: still one infra server each.